Aalto University
School of Science

Stream Processing and Big Data Platforms

Hong-Linh Truong
Department of Computer Science
linh.truong@aalto.fi

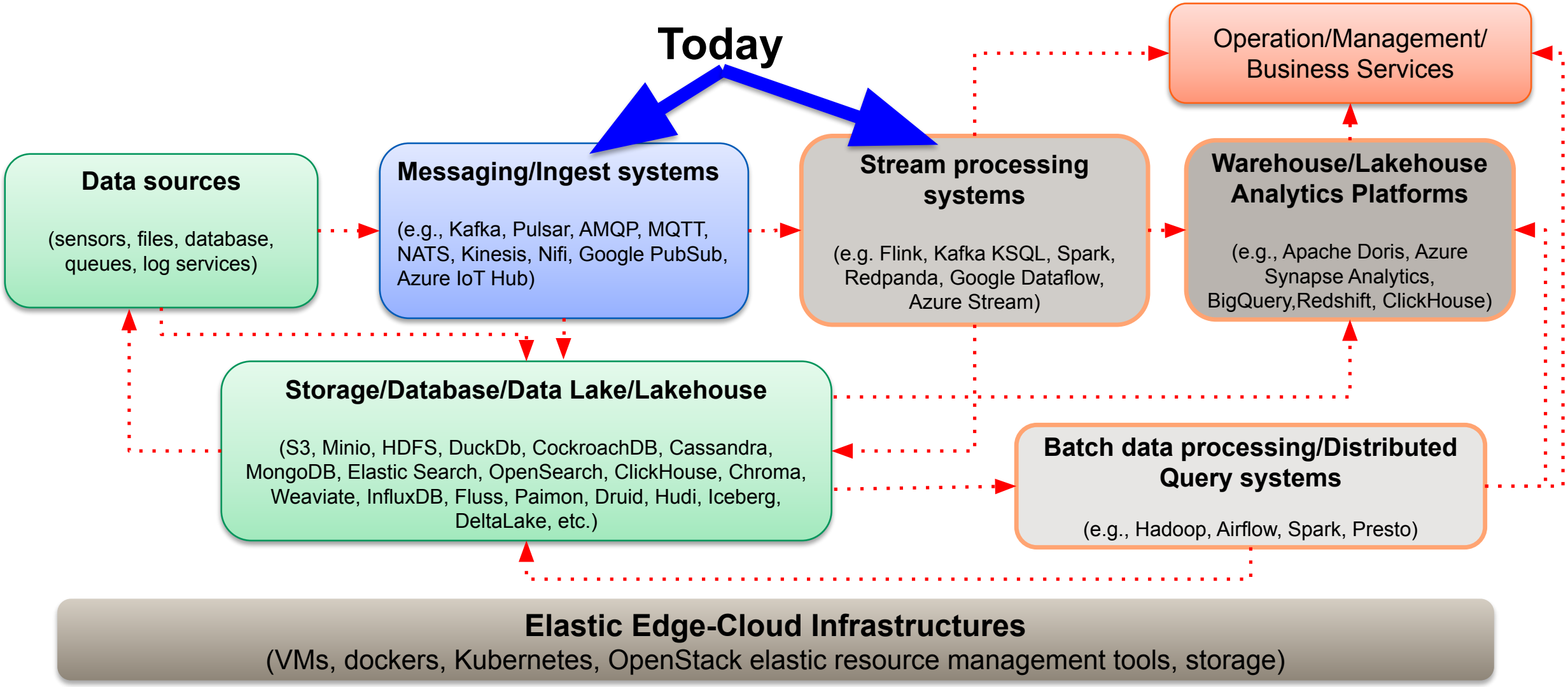
04.3.2026

Content is available under
CC BY-SA 4.0 unless otherwise stated

Learning objectives

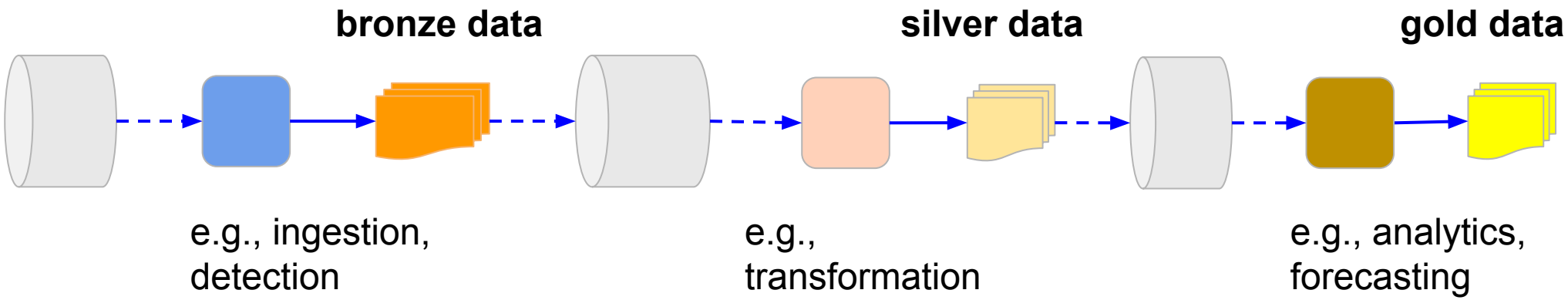
- Understand fundamental concepts and techniques in stream processing in big data platforms
- Able to design basic streaming analytics in big data platforms and applications
- Able to select and use common stream processing frameworks

Our big data at large-scale: the big picture in this course



Stream analytics for data in motion

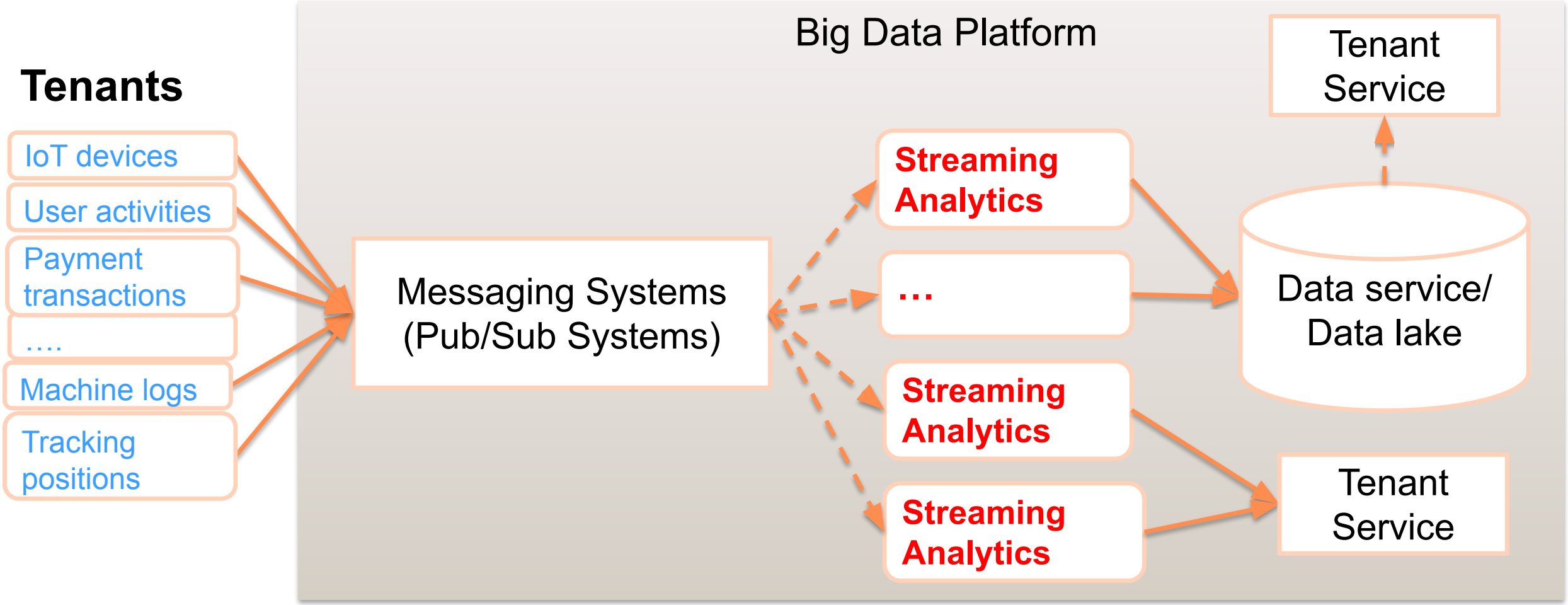
- Fast processing of requests or reaction to events: *e.g., payments, orders, and failures*
- Basic data processing: drop, merge/join, cleaning and transformation
- Advanced fast analytics for windowed data
- Real-time AI/ML inferences with windowed data



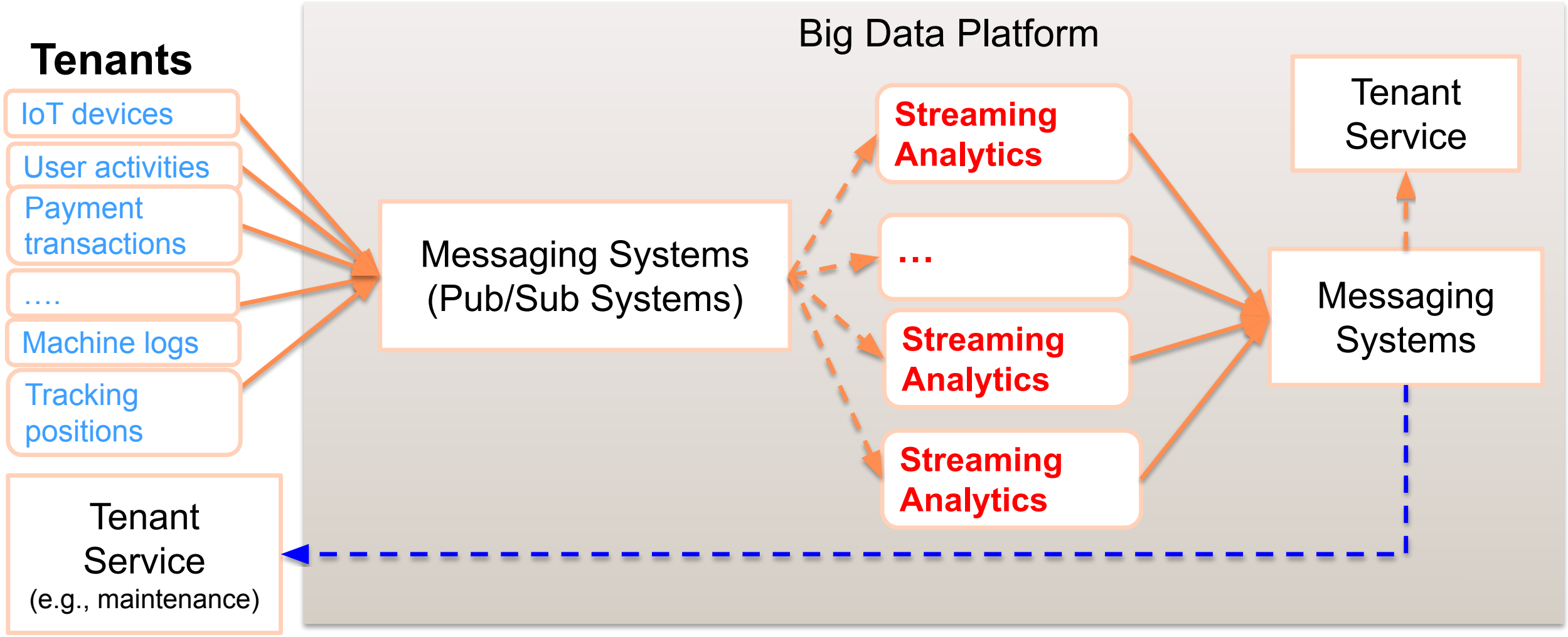
Stream processing in big data

- Big data from streams at near real-time
  - the data element/unit may be “small” but voluminous and delivered in a near real-time manner
  - high and volatile throughput, but extreme low processing time expected
  - not just *“take a record and store it into a database”*
- Processing fundamentals
  - *task parallelism*: multiple tasks for processing data
  - *data parallelism*: data is partitioned into concurrent/parallel data streams ⇒ distributed, parallel processing tasks
  - *stateful analytics:* processing needs state information across multiple data records and time

Near real-time streaming data processing (1)



Near real-time streaming data processing (2)



Stream processing and big data platforms

- Stream processing services as a component of data platforms
  - a big data technology for pre-processing, ingestion and high-level analytics, including near-real time ML
- Stream processing services as data platforms
  - a big data platform offers mainly stream processing services for streaming analytics
  - analytics on the fly as the first class feature
  - e.g., IoT analytics, e-commerce user activities, fraud detection, real time AI/ML

Stream Processing – key concepts and components

Common building blocks

- **The way to connect data sources to streams and obtain data** records (**messages) from the streams**
  - focusing very much on *connector concepts* and well-defined message structures (JSON, Avro, customized binary format, etc.)
  - connectors implement complex data handling mechanisms (low level session management, message retainment, delivery quality of service)
- **The way to specify/program the "analytics" logic**
  - *analytics functions, statements* and how they are glued together to process flows of messages
  - high-level, easy to use
- **The distributed engine to process analytics tasks**
  - handle complex task processing atop multiple compute nodes
- **The way to push the result to external components** (sink databases, new streams, files)

Data stream programming

Data stream: a sequence/flow of data units

Data units are defined by applications: a data unit can be data described by a primitive data type or by a complex data type, a serializable object, etc.

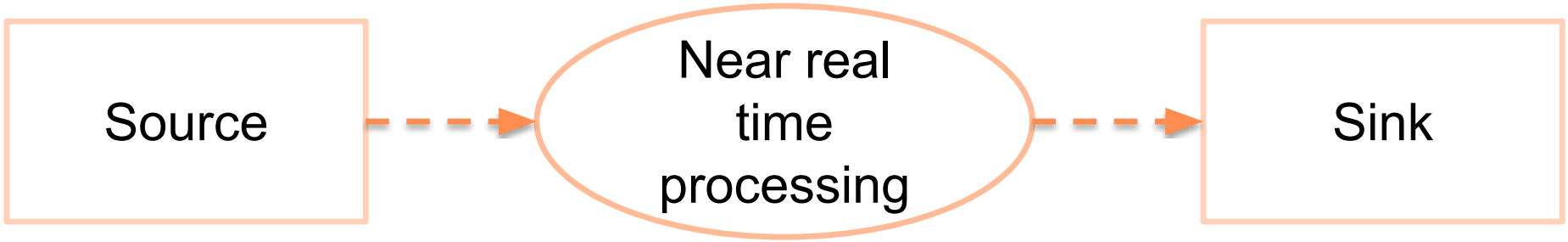
Streaming data: produced by (near)real time data sources as well as (big) static data sources ⇒ *unbounded* and *bounded*

Examples of data streams

- Continuous media (e.g., video for video analytics)
- Discrete media (e.g., stock market events, twitter events, system monitoring events, comments, notifications, log records)

Messages of events/data records

- Messages encapsulating real-world events, data records and other types of data
- Data to be sent/processed can be in a simple or complex structure



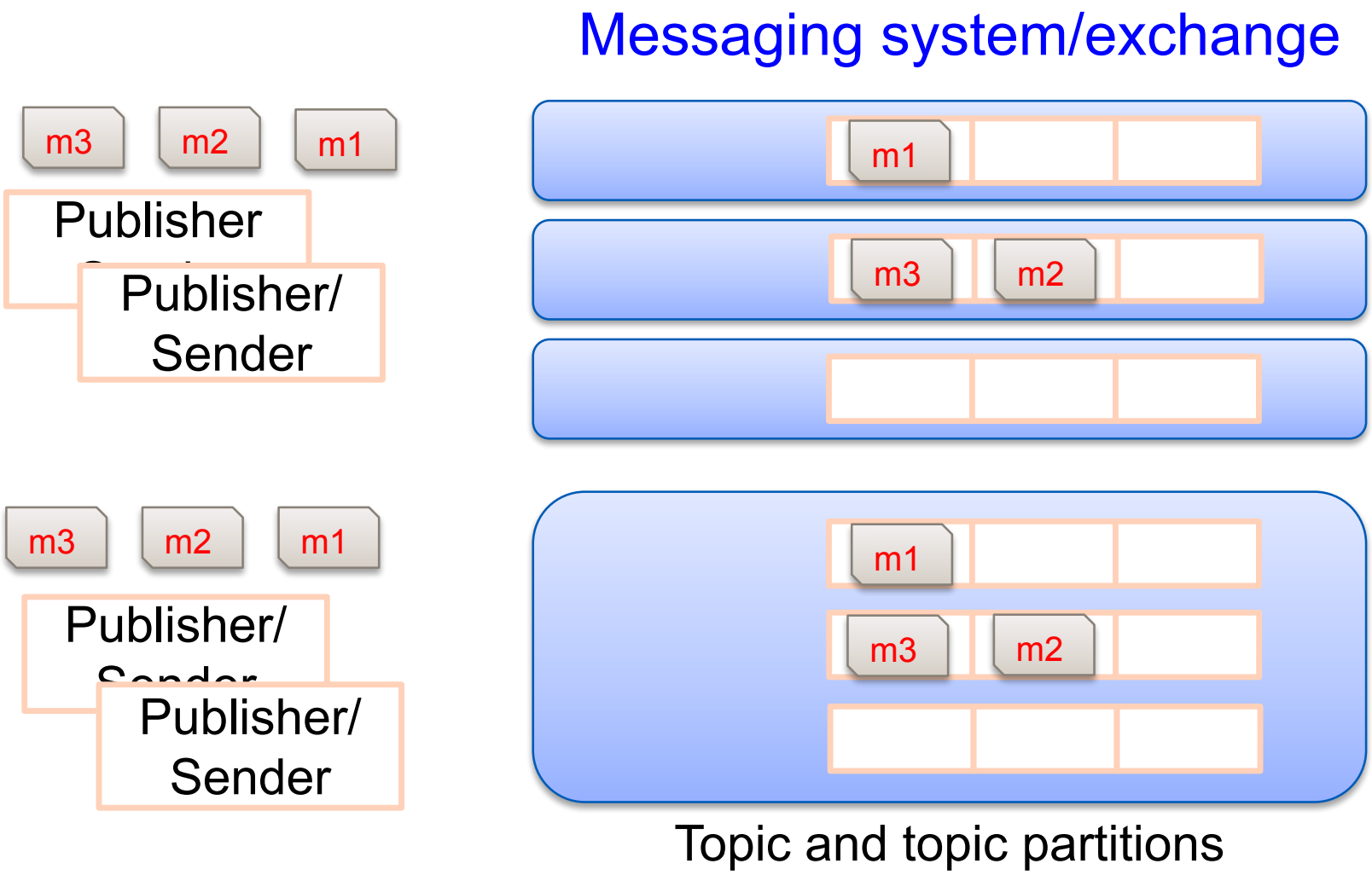
We focus on unbounded discrete messages of data

a table in a database/file can be seen as "bounded data"

Message representations and streams

- Data Sources
  - via messaging systems, databases, websocket, different IO adapters/connectors, etc.
- Data Sinks
  - messaging systems, databases, file storage/systems (S3, HDFS), etc.
- Data representations
  - text/CSV, JSON, Arvo format, etc.
  - serialization and deserialization (short name: SerDe) are required
  - data format validation
  - data schema registry for registered schemas

Data message exchange strategies (1)



topic=queue; no partition

E.g., AMQP

```
channel.basic_publish(
    exchange=args.exchange, routing_key=args.queuename, body=message
)
```

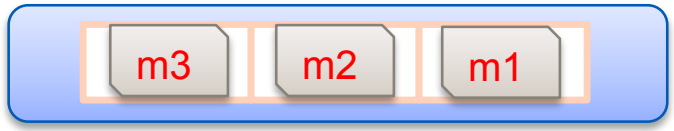
topic = n partitions = n queues

E.g., Kafka confluent python producer API:

```
produce()
    produce(topic [, value] [, key] [, partition] [, on_delivery] [, timestamp] [, headers])
```

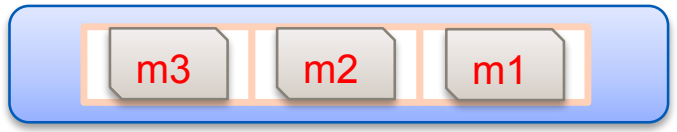
Data message exchange strategies (2)

Messages in systems



simple straight forward

E.g., using default AMQP exchange



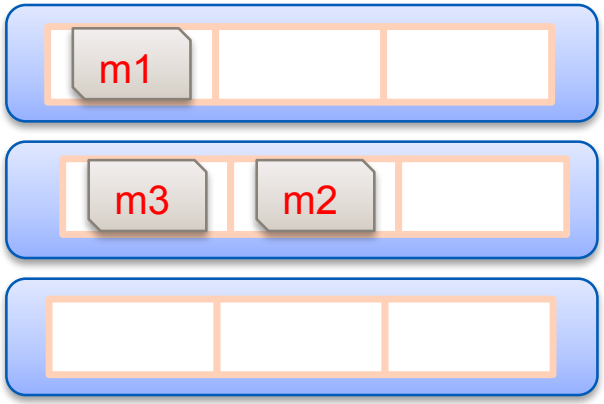
fant-out/broadcast

allowing parallel processing of the same messages

E.g., Kafka consumer groups

different mechanisms for "routing"

complex mapping/routing; key-based routing, round-robin, etc.



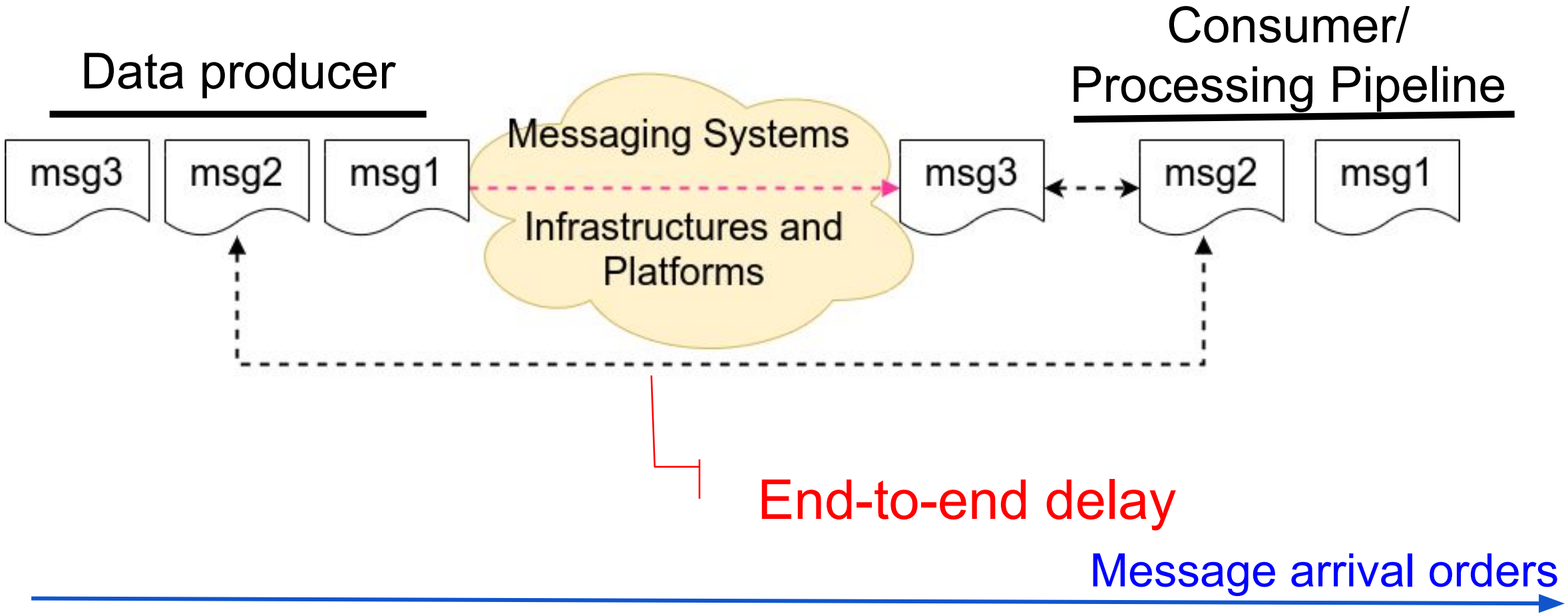
parallel/ concurrent processing of different messages

E.g., using AMQP topic exchange or Pulsar Function

Key issues in streaming data (1)

- Data order & delivery
  - late data, out of order data
- Times associated with messages and processing
- Data parallelism
  - key-based data processing
- Task parallelism
  - stateful vs stateless processing

Key issues in streaming data (2)

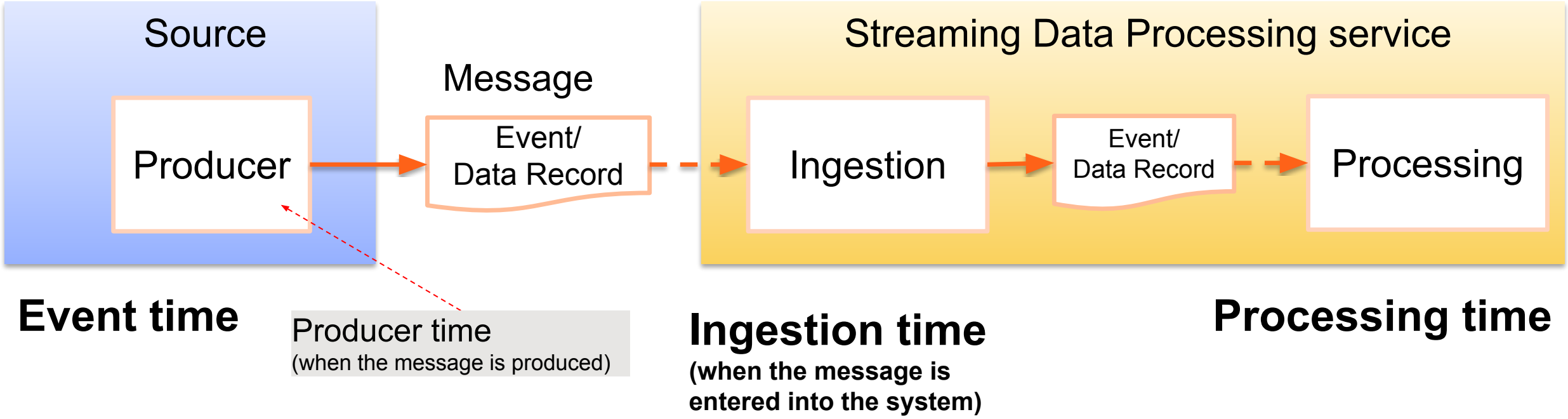


Without a timestamp associated to a message, do we know the delay or out of order?

What is the consequence of delay/out of order for processing?

Key issues in streaming data (3): the notion of times

Times associated with data and processing



Which time is important for analytics (from a business viewpoint)?

Example

Topics with Producer time and with messaging system time

```
topics = [
    # CREATE_TIME producer code sets the timestamp of the message
    NewTopic('bts_create_time', num_partitions=1, replication_factor=1,
             config={'message.timestamp.type': 'CreateTime'}),

    # LogAppendTime broker sets/overwrites the timestamp of the message
    NewTopic('bts_log_append', num_partitions=1, replication_factor=1,
             config={'message.timestamp.type': 'LogAppendTime'}),
]
```

Event has no event time

```
payload={
    'station_id': '1161115016',
    'datapoint_id': 121,
    'alarm_id': 308,
    'value': 240
}
```

```
station_id        : 1161115016
Kafka ts type     : CREATE_TIME
Kafka ts          : 2026-03-03 13:34:36.745000
Payload ts        : NONE
```

Flexible way to set message timestamp

```
payload={
    'station_id': '1161114050',
    'datapoint_id': 143,
    'alarm_id': 312,
    'event_time':'2017-02-18 18:56:20',
    'value': 28.5
}
```

```
station_id        : 1161115040
Kafka ts type     : CREATE_TIME
Kafka ts          : 2026-03-03 13:38:33.645000
Payload ts        : 2017-02-18 18:22:03
Kafka vs Payload: DRIFT = 285102990.6s
```

```
payload={
    'station_id': '1161114008',
    'datapoint_id': 121,
    'alarm_id': 308,
    'event_time': '2017-02-18 18:34:09',
    'value': 240
},
```

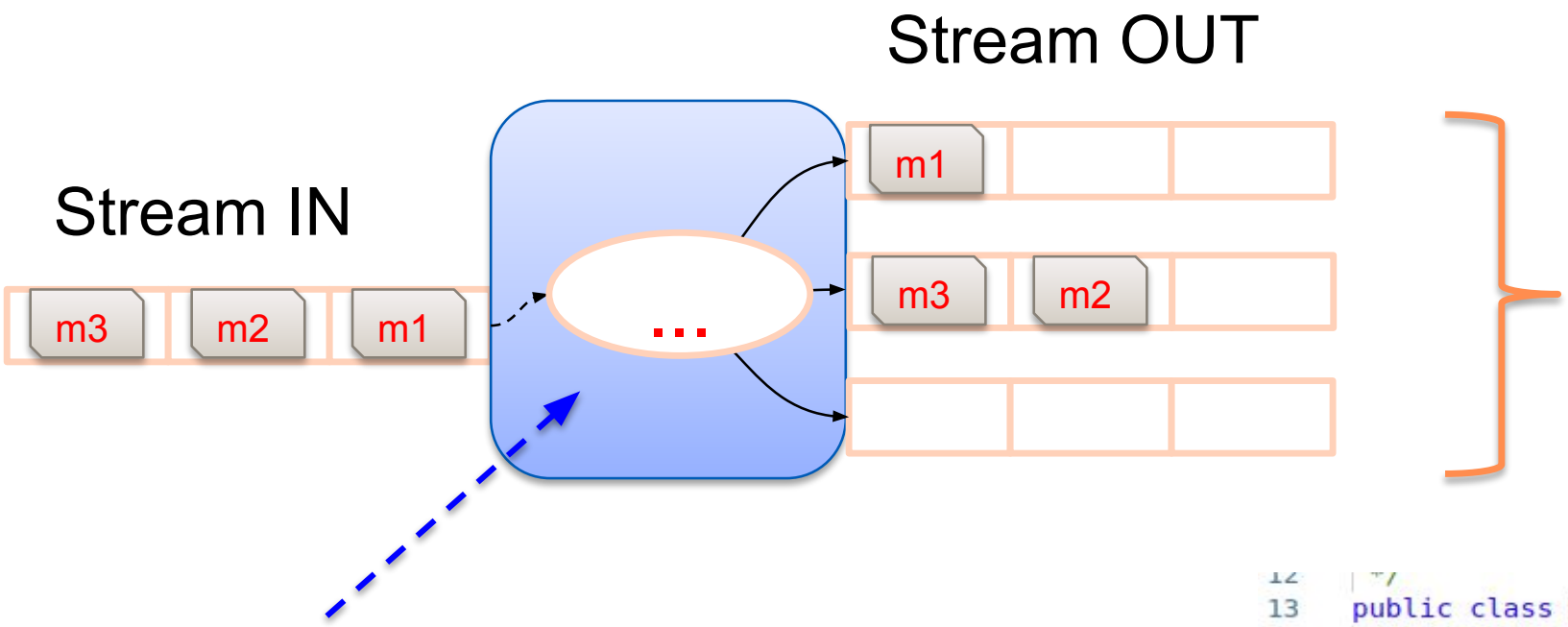
```
station_id        : 1161114008
Kafka ts type     : CREATE_TIME
Kafka ts          : 2017-02-18 18:34:09
Payload ts        : 2017-02-18 18:34:09
Kafka vs Payload: THE SAME
```

Message time set by the system

```
payload={
    'station_id': '1161115040',
    'datapoint_id': 141,
    'alarm_id': 312,
    'event_time': '2017-02-18 18:20:49',
    'value': 56
},
```

```
station_id        : 1161115040
Kafka ts type     : LOG_APPEND_TIME
Kafka ts          : 2026-03-03 13:38:35.640000
Payload ts        : 2017-02-18 18:20:49
Kafka vs Payload: DRIFT = 285103066.6s
```

Data parallelism: partition data based on some keys for analytics



Can be processed in parallel using multiple workers

With keyed data: enable parallel processing based on the keys (partitioning)

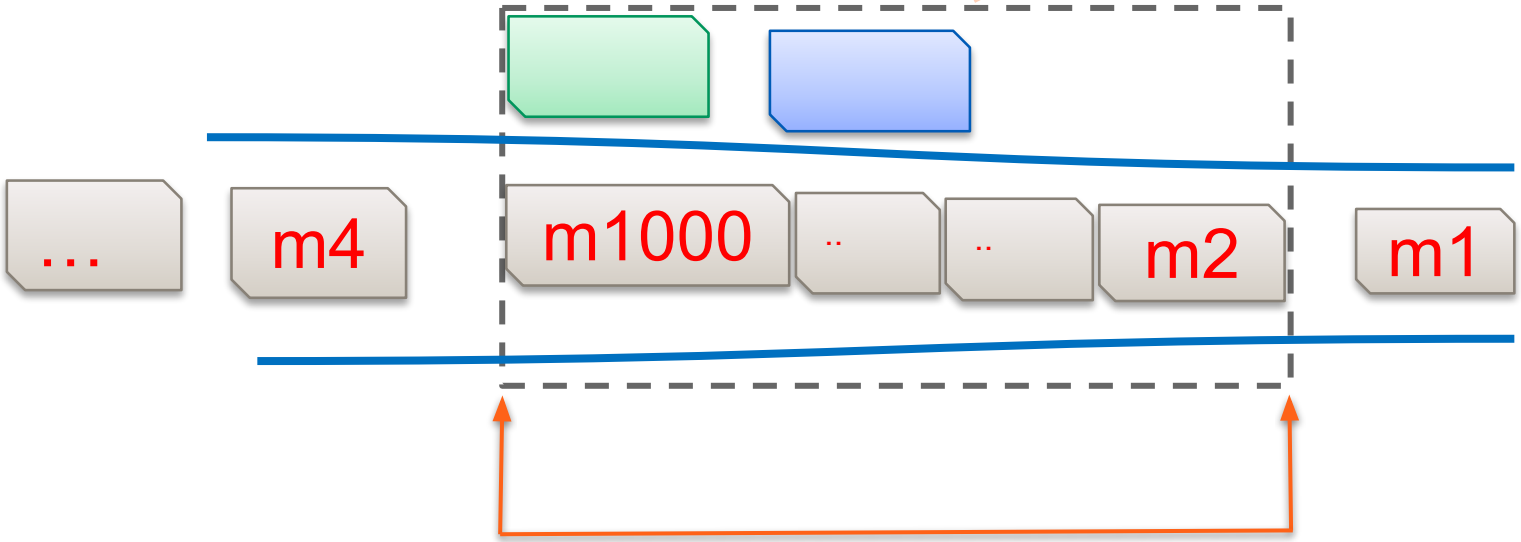
```
public class BTSAlarmEvent {
    public String station_id;
    public String datapoint_id;
    public String alarm_id;
    public Date event_time;
    public float value;
    public float valueThreshold;
    BTSAlarmEvent() {
```

keys?

Windows of data

Window is used to group data for processing:

Which constraints are used to determine a window?



a stream of events

sliding/tumble window size: a period of time or number of events/records

Message arrival order

Windowing

- Windows size
  - time or number of records
- Tumbling Window
  - identified by size, no gap between windows
- Sliding Window
  - identified by size and a sliding interval (slide)
- Session Window
  - identified by “gap” between windows (e.g., the gap of events is used to mark “sessions”)

Functions applied to windows of data

If we

- specify a set of conditions ⇒ windows will be created according to the conditions to store message in corresponding windows

then we can

- Apply functions to messages in the window that match these conditions

Task parallelism: we can have a lot of such functions executed in parallel in multiple compute nodes

Functions

- Can be simple or complex!
  - built-in and user-defined functions
- Core for analytics and ML
- Examples
  - individual threshold/alarm based alerting, atypical events monitoring
  - data rollup
  - anomaly detection based on statistical functions, like quantile/T-digest, …
  - real time AI/machine learning

Example

Monitoring working hours of (taxi/truck) drivers (assume events about pickup/drop captured at near real-time):

- Windows: 12 hours
- Partitioning data/Keyed streams: licenseID
- Function: determine working and break times and check with the law/regulation

Source:
https://www.infoworld.com/article/3293426/how-to-build-stateful-streaming-applications-with-apache-flink.html

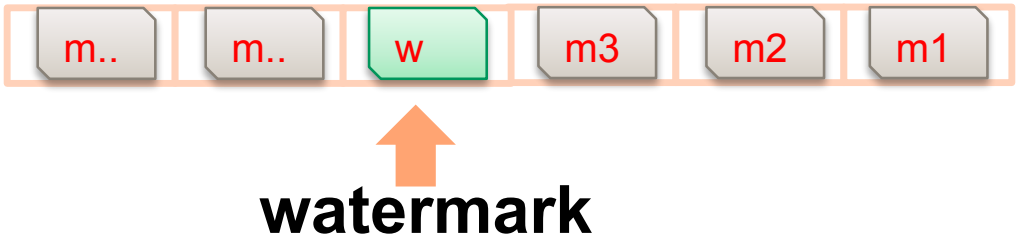
What if events/records come late into the windows?

Do we need to deal with late, out of order events/records?

correctness and completeness issues

Support lateness

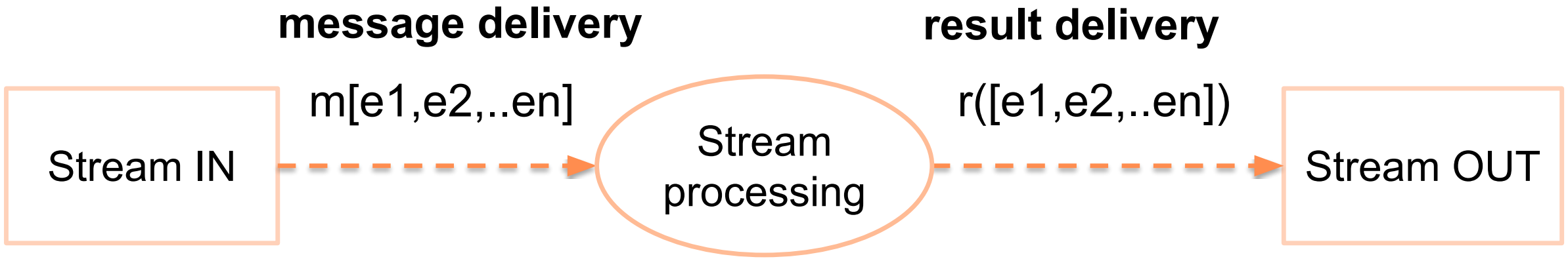
- Identify timestamp of events/data records
- Identify watermark in streams
  - a watermark is a timestamp
  - a watermark indicates that no events which are older that the watermark should be processed
  - enable the delay of processing functions to wait for late events
- Using watermark to ignore late data ⇒ computing under "incompleteness assumption"



Delivery guarantees

At least once? or at-most-once?

Exactly once? End-to-end exactly once?



What if the stream processing fails and restarts?

Message and processing guarantees

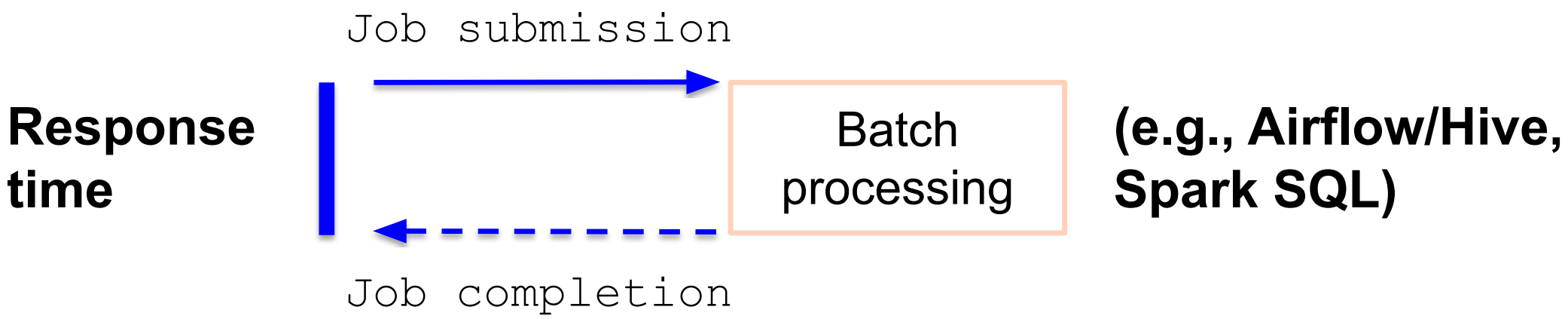
- Message guarantees are the job of the broker/messaging system
- Processing guarantees are the job of the stream processing frameworks
- They are highly connected if messaging systems and processing frameworks are tightly coupled (e.g., Kafka case)

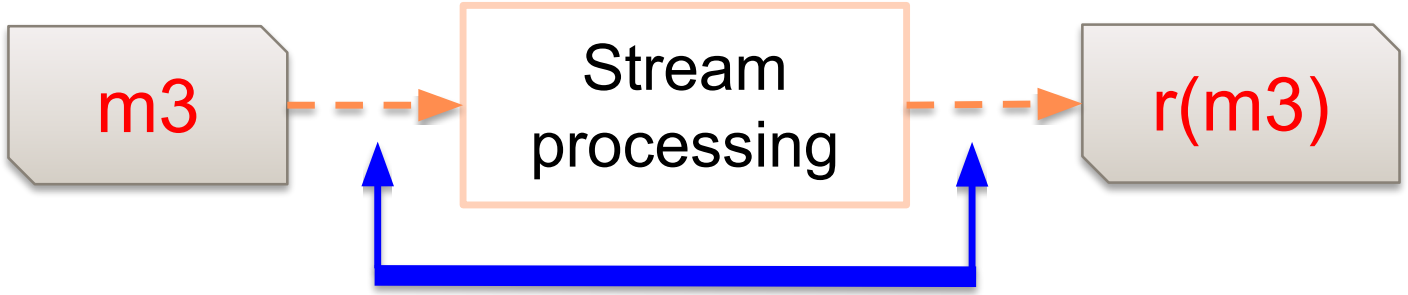
End-to-end exactly once

- Exactly once for *only processing* is not enough
- Messaging systems must support
  - redeliver messages/data, recoverable data
- Sink and output must support exactly one
  - idempotent results, roll back
- Coordination among various components

Further reading:
https://flink.apache.org/features/2018/03/01/end-to-end-exactly-once-apache-flink.html
https://www.confluent.io/blog/simplified-robust-exactly-one-semantics-in-kafka-2-5/
https://docs.microsoft.com/en-us/azure/hdinsight/spark/apache-spark-streaming-exactly-once

Performance metrics





Service time/latency

Service time/latency and throughput

- Service time/latency
  - **subseconds**! e.g., milliseconds
  - max, min or percentile? ⇒ up to application requirements
- Throughput
  - how many messages can be processed per second?
- Goal: **low latency and high throughput**!

Structure of streaming data processing programs (1)

- We have multiple streams of data, different functions for processing data, multiple computing nodes
- Data exchange between tasks
  - links in task graphs reflect data flows
- Stream processing
  - centralized or distributed (in terms of computing resources)
  - simple functions vs complex ones

Structure of streaming data processing programs (2) - examine a simple example

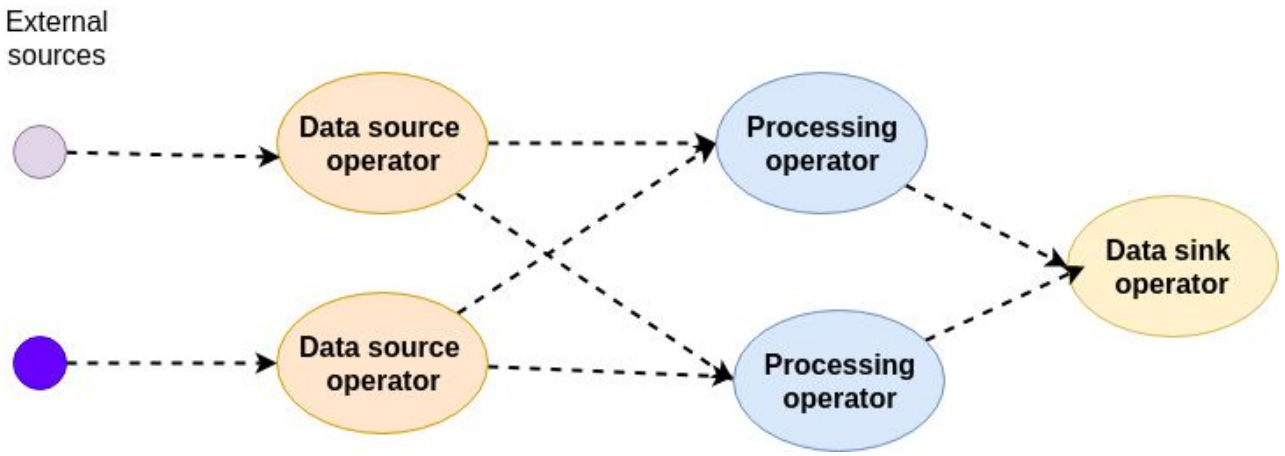
```
'''
WAIT AND PROCESS DATA
'''
while True:
    '''
    Receive the data from source
    '''
    msg = consumer.receive()
    '''
    when should we do this?
    consumer.acknowledge(msg)
    '''
    try:
        '''
        MAIN TRANSFORMATION, HERE IS WITH A FUNCTION
        '''
        ## assume that the selected data schema is json
        result =dt_process_json_style(msg,op_processor)
        ##store the result to the right data sink
        dt_store_to_sink(result)

    except Exception as ex:
        logging.warn(f'{ex}')
        logging.info("Continue to wait")
```

How to handle possible errors in these situations?

Note: Example with an external Pulsar consumer for data transformation

Structure of streaming data processing programs (3)

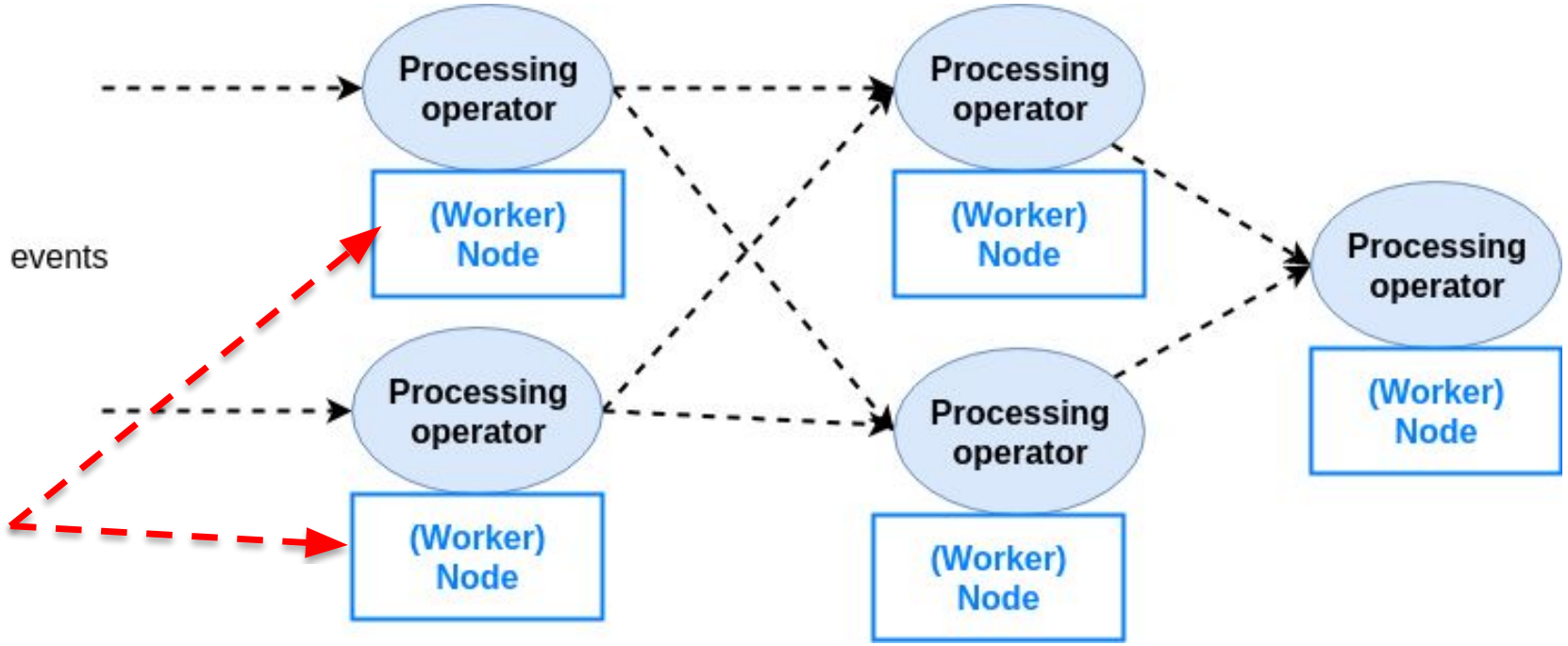



```
//      parse the data, determine alert and return the alert in a json string
//      Apply function on data stream
        DataStream<String> alerts = btsdatastream
//              .flatMap(new BTSParser()
                .flatMap(new BTS_Trend_Parser()
                 /*
                     Another example is to have:
                     new FlatMapFunction<String, BTSAlarmEvent>() {
                         @Override
                         public void flatMap(String valueString, Collector<BTSAlarmEvent> out) {
                                 String[] record = valueString.split(",");
                                 ....
                             out.collect(...);
                            }
                 })
                */
                )
                //.setParallelism(5)    // uncomment this line to scale the Parser stream and set the valu
                .keyBy(new AlarmKeySelector()
                     /* another way is to have:
                     new KeySelector<BTSAlarmEvent, String>() {
                       public String getKey(BTSAlarmEvent btsalarm) { return btsalarm.station_id; }
                    }
                */
                )
                .window(SlidingProcessingTimeWindows.of(Time.seconds(60), Time.seconds(5)))
//              .window(SlidingEventTimeWindows.of(Time.minutes(5), Time.seconds(5))) // set the window si
//              .process(new MyProcessWindowFunction()).setParallelism(1);
                .process(new TrendDetection()).setParallelism(1);
        //.setParallelism(5);  // uncomment this line to scale the stream processing and set the value for
```

- Dataflows:
  - Data source operators: represent sources of streams
  - Processing operators: represent processing functions

Distributed processing topology in many compute nodes

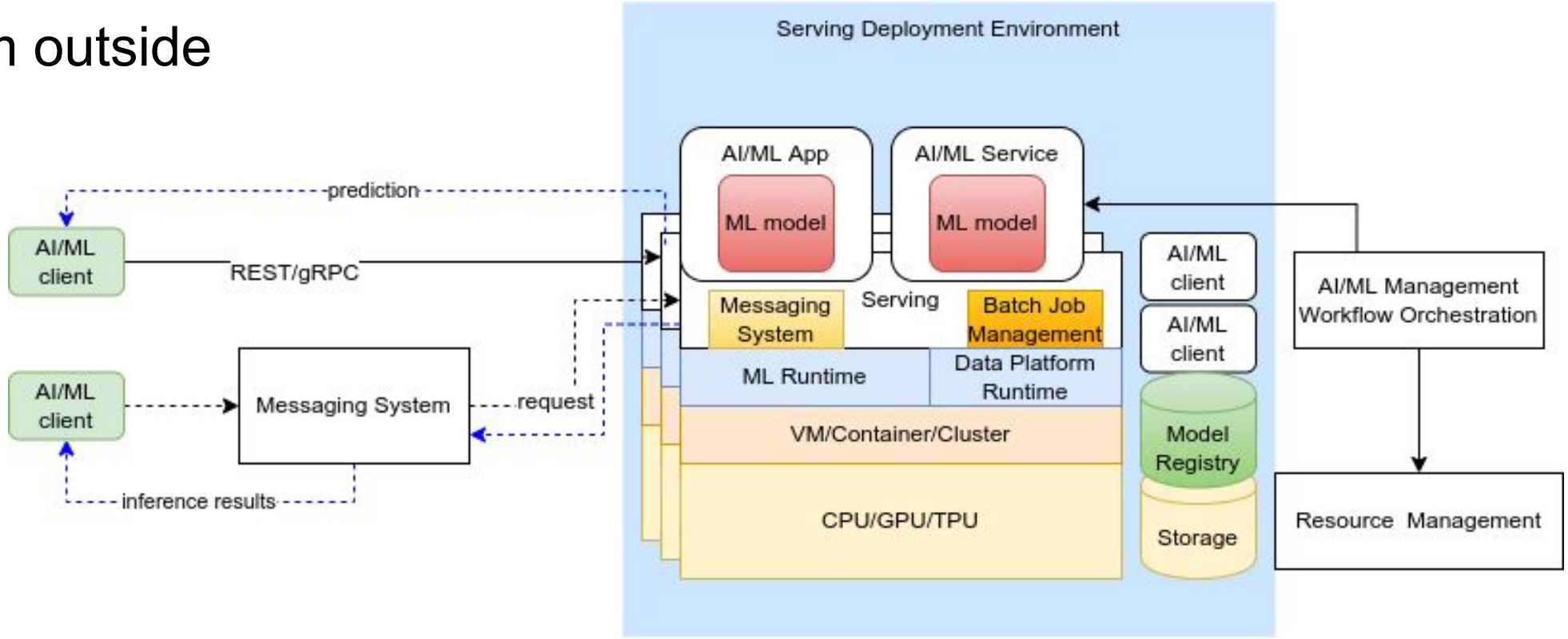
A graph of tasks (running operators); all tasks are running; data messages are exchanged based on different strategies (forward, key-based, round-robin, union, broadcast, random)



Nodes of a cluster (VMs, containers, Kubernetes)

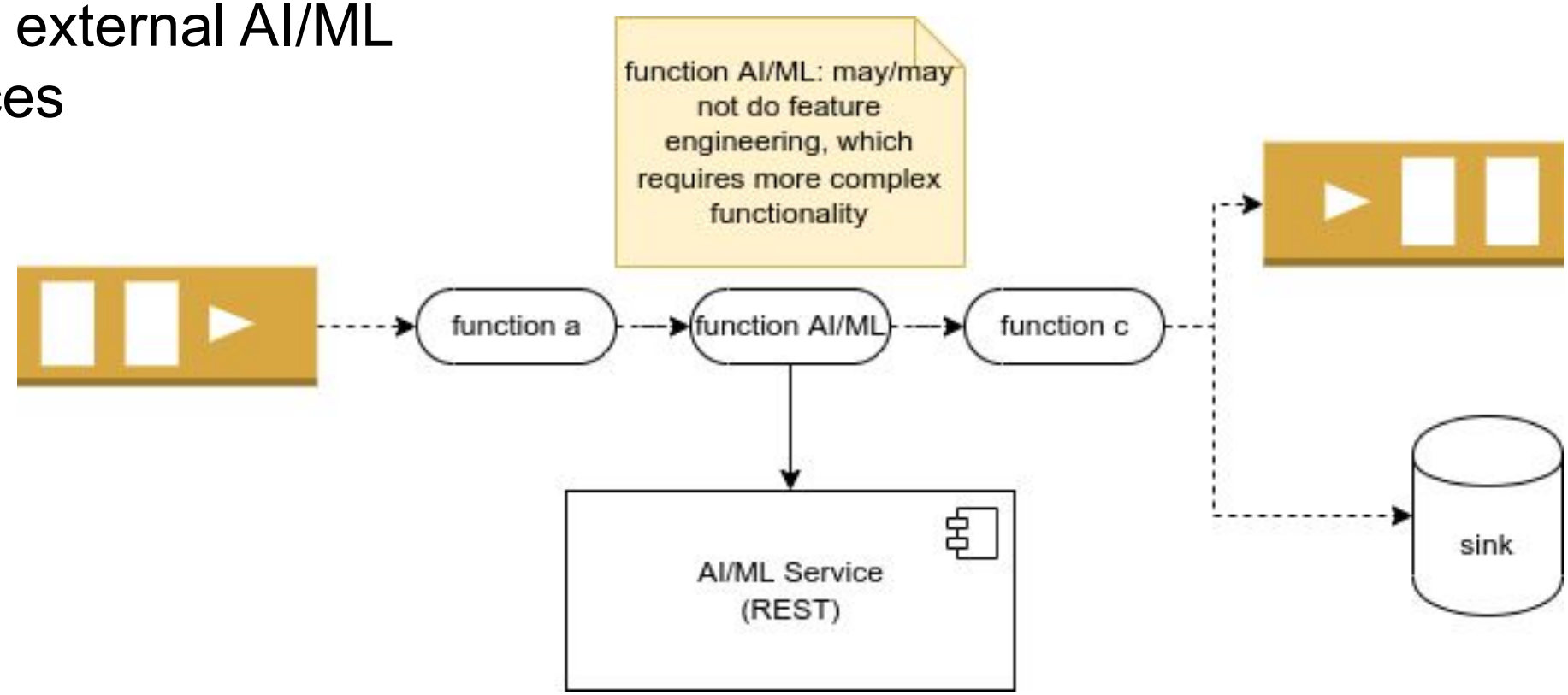
Integration with AI/ML (1)

From outside



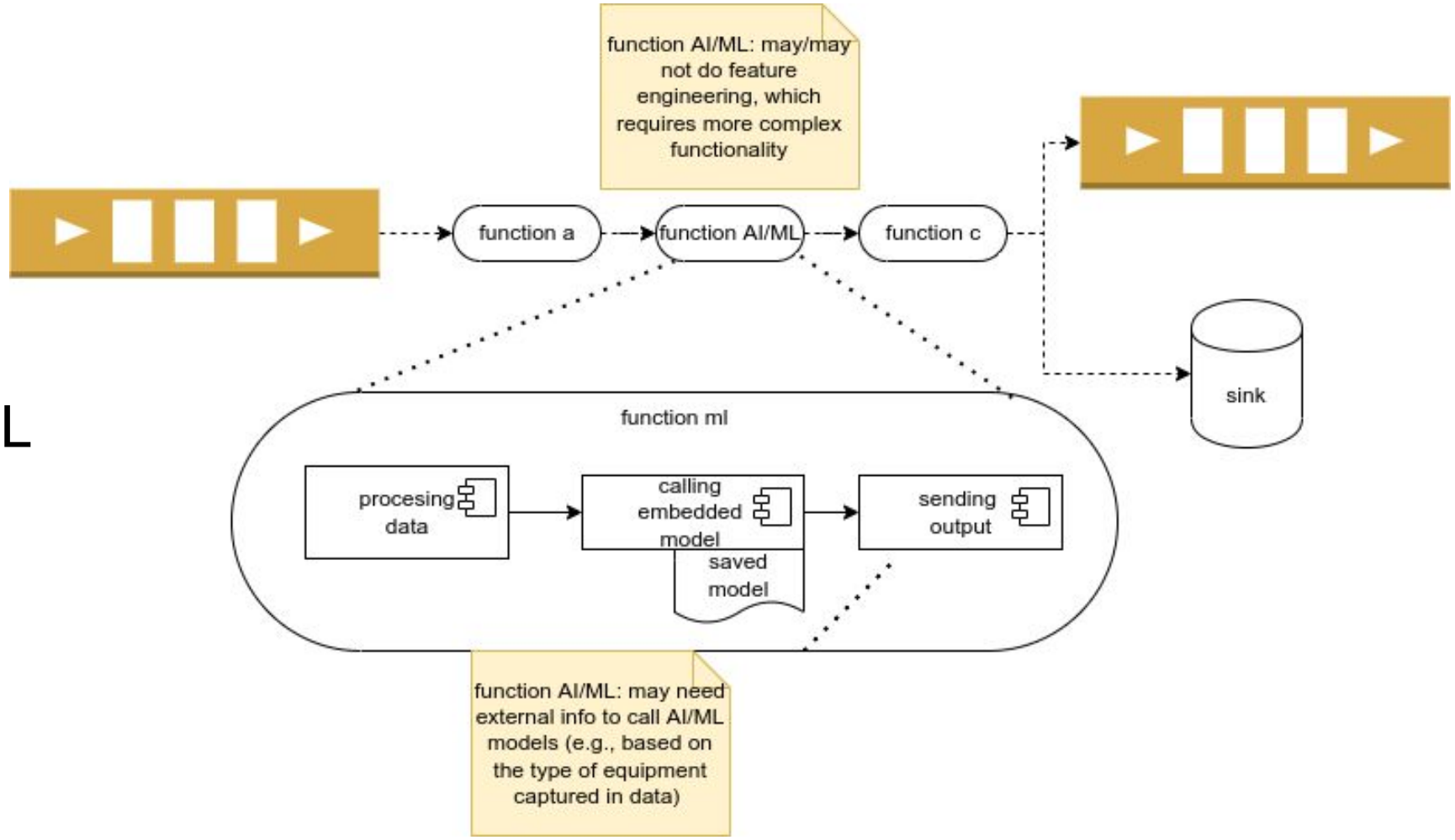
Integration with AI/ML (2)

Using external AI/ML services

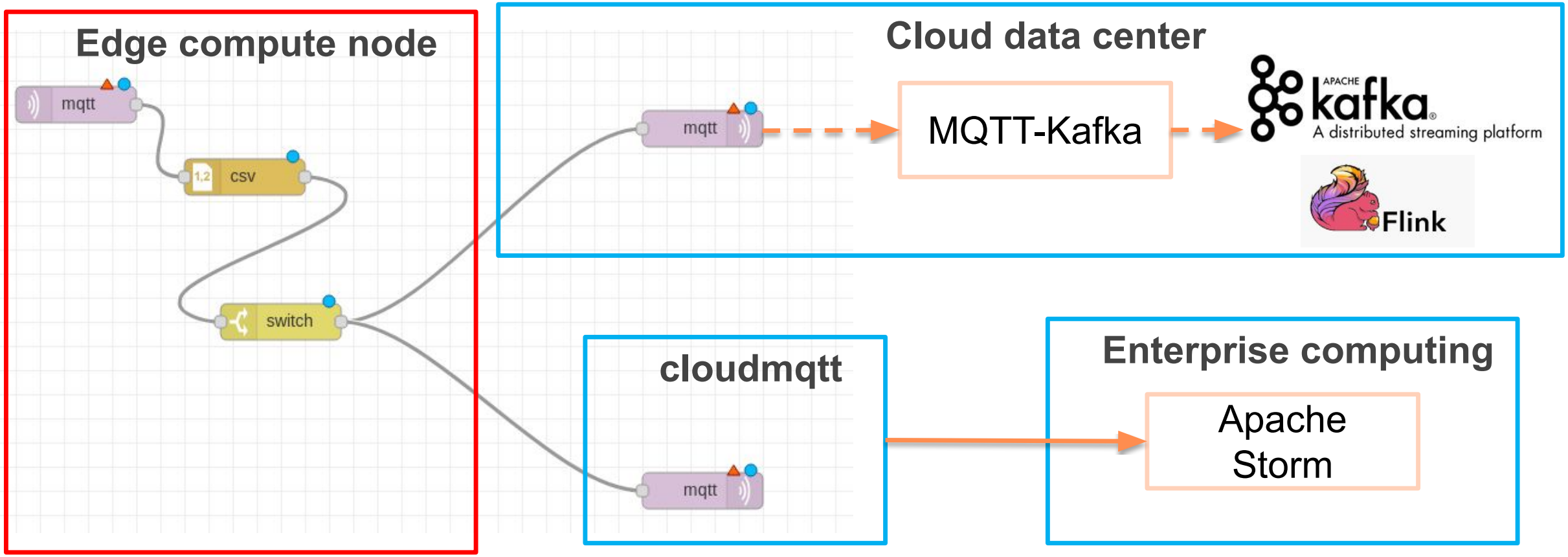


Integration with AI/ML (3)

Embedded AI/ML capabilities



Distributed, composable topologies in cross distributed sites



Existing frameworks - programming level (1)

- How to write streaming program?
- With programming languages
  - low level APIs
  - DSL
  - Java, Scala, Python (Spark, Flink, Kafka)
- High-level data models
  - KSQL
- Flow/pipeline description
  - Node-RED/GUI-based flow editors

Existing frameworks - programming level (2)

- Connector library for data sources/sinks
  - very important for application domains
- Runtime elasticity
  - add/remove (new) operators
  - add/remove underlying compute nodes
- Fault tolerance support

Where do you find most of concepts that we have discussed

- Apache Storm
  - https://storm.apache.org/
- Apache Spark (Structured Streaming)
  - https://spark.apache.org/
- Apache Kafka Streams and KSQL
  - strongly bounded to Kafka messaging
- **Apache Flink (Stream Analytics)**
  - **native, clustered, better data sources/sinks**
- Apache Beam (https://beam.apache.org/)
  - unifying programming models for batch and stream processing

Practical learning paths

- **Path 1**: if you don’t have a preference and need challenges
  - Apache Flink Stream API (e.g., with RabbitMQ/Kafka connectors)
- **Path 2**: many of you have worked with Kafka
  - Kafka Streams DSL (everything can be done with Kafka)
- **Path 3**: for those of you who are working with Spark (and Python is the main programming language)
  - Apache Spark Structured Streaming
- **Path 4**: for those who deal with MQTT brokers
  - Apache Storm (but also Kafka, …): Spout and Bolt API or Stream API

Summary

- **Focus:**
  - Practical programming with one of the stacks:
    - Apache Flink (with different connectors)
    - Kafka Streams
  - Check the common concepts in other tools/systems
- **Action:**
  - Work on use cases where you can use stream analytics (as a user/developer) ⇒ there are many interesting analytics
  - Provision services for stream processing (as a platform)

Thanks!

Hong-Linh Truong
Department of Computer Science

rdsea.github.io

A!

—

Kiitos
aalto.fi